

ROMAN NUMERALS & ALGORITHM NOTES

1. Basic Roman Numeral Symbols

Object / Dictionary definition of base symbols:

obj = { I: 1, V: 5, X: 10, C: 100, D: 500, M: 1000 }

Units	Tens	Hundreds	Thousands
I (1)	X (10)	C (100)	M (1000)
II (2)	XI (11)	CC (200)	MM (2000)
III (3)	XII (12)	CCC (300)	MMM (3000)
IV (4)	XIII (13)	CD (400)	-
V (5)	XIV (14)	D (500)	-
VI (6)	XV (15)	DC (600)	-
VII (7)	XVI (16)	DCC (700)	-
VIII (8)	XVII (17)	DCCC (800)	-
IX (9)	XIX (19)	CM (900)	-
X (10)	XX (20)	M / XC	-

2. Conversion Rules & Examples

Rule (Bottom-Up / Subtraction): When a smaller numeral symbol precedes a larger one, subtraction is applied.

IV = 5 - 1 = 4

XL = 50 - 10 = 40

$$XC = 100 - 10 = 90$$

Composition Rule: A number containing two or more decimal digits is formed by appending the Roman numeral for each place value. A missing place represented by zero is omitted.

$$CLX = 100 + 60 = 160$$

$$CCVII = 200 + 5 + 2 = 207$$

$$MLXVI = 1000 + 50 + 10 + 5 + 1 = 1066$$

$$XXVII = 10 + 10 + 5 + 1 + 1 = 27$$

$$XXXIX = XXX + (IX: 10 - 1) = 39$$

$$CCXLVI = 200 + (XL: 40) + (VI: 6) = 246$$

3. Algorithmic Processing Logic

Evaluating Roman numerals programmatically (Top-to-Bottom approach):

```
if (nextval > current[obj[i]]) {  
    result -= obj[current];  
} else {  
    result += obj[current];  
}
```

Tracing Example for *XXVI*:

result = 10 + 10 + 5 + 1 = 26

i = 0: current = X (10), next = obj[arr[1]] = X (10). Comparison: $10 < 10$ (False) → result += 10

i = 1: current = X (10), next = obj[arr[2]] = V (5). Comparison: $10 < 5$ (False) → result += current (10)

i = 2: current = V (5), next = I (1). Comparison: $5 < 1$ (False) → result += 5

i = 3: current = I (1), next = none → result += 1